

File handling in C

Basics of File Handling in C

File handling in C is the process in which we create, open, read, write, and close operations on a file.

C language provides different functions such as `fopen()`, `fwrite()`, `fread()`, `fseek()`, `fprintf()`, etc. to perform input, output, and many different C file operations in our program.

Why do we need File Handling in C?

So far the operations using the C program are done on a prompt/terminal which is not stored anywhere. The output is deleted when the program is closed. But in the software industry, most programs are written to store the information fetched from the program. The use of file handling is exactly what the situation calls for.

In order to understand why file handling is important, let us look at a few features of using files:

- **Reusability:** The data stored in the file can be accessed, updated, and deleted anywhere and anytime providing high reusability.
- **Efficient:** A large amount of input may be required for some programs. File handling allows you to easily access a part of a file using few instructions which saves a lot of time and reduces the chance of errors.
- **Storage Capacity:** Files allow you to store a large amount of data without having to worry about storing everything simultaneously in a program.

Types of Files in C

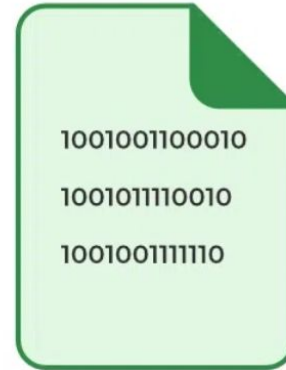
A file can be classified into two types based on the way the file stores the data. They are as follows:

- **Text Files**
- **Binary Files**

Types of Files in C



file.txt



file.bin

1. Text Files

A text file contains data in the **form of ASCII characters** and is generally used to store a stream of characters.

- Each line in a text file ends with a new line character ('\n').
- It can be read or written by any text editor.
- They are generally stored with **.txt** file extension.
- Text files can also be used to store the source code.

2. Binary Files

A binary file contains data in **binary form (i.e. 0's and 1's)** instead of ASCII characters. They contain data that is stored in a similar manner to how it is stored in the main memory.

- The binary files can be created only from within a program and their contents can only be read by a program.
- More secure as they are not easily readable.
- They are generally stored with **.bin** file extension.

File Pointer in C

A file pointer is a reference to a particular position in the opened file. It is used in file handling to perform all file operations such as read, write, close, etc. We use the **FILE** macro to declare the file pointer variable. The FILE macro is defined inside **<stdio.h>** header file.

Syntax of File Pointer

FILE* *pointer_name*;

File Pointer is used in almost all the file operations in C.

C File Operations

C file operations refer to the different possible operations that we can perform on a file in C such as:

1. Creating a new file – **fopen()** with attributes as “a” or “a+” or “w” or “w+”
2. Opening an existing file – **fopen()**
3. Reading from file – **fscanf()** or **fgets()**
4. Writing to a file – **fprintf()** or **fputs()**
5. Closing a file – **fclose()**

File operation	Declaration & Description
fopen() - To open a file	Declaration: FILE *fopen (const char *filename, const char *mode) fopen() function is used to open a file to perform operations such as reading, writing etc. In a C program, we declare a file pointer and use fopen() as below. fopen() function creates a new file if the mentioned file name does not exist. <pre>FILE *fp; fp=fopen ("filename", "'mode");</pre> Where, fp - file pointer to the data type "FILE". filename - the actual file name with full path of the file. mode - refers to the operation that will be performed on the file. Example: r, w, a, r+, w+ and a+. Please refer below the description for these mode of operations.
fclose() - To close a file	Declaration: int fclose(FILE *fp); fclose() function closes the file that is being pointed by file pointer fp. In a C program, we close a file as below. <pre>fclose (fp);</pre>
fgets() - To read a file	Declaration: char *fgets(char *string, int n, FILE *fp) fgets function is used to read a file line by line. In a C program, we use fgets function as below. <pre>fgets (buffer, size, fp);</pre> where, buffer - buffer to put the data in. size - size of the buffer fp - file pointer
fprintf() - To write into a file	Declaration: <pre>int fprintf(FILE *fp, const char *format, ...);</pre> fprintf() function writes string into a file pointed by fp. In a C program, we write string into a file as below. fprintf (fp, "some data"); or <pre>fprintf (fp, "text %d", variable_name);</pre>

Open a File in C

For opening a file in C, the `fopen()` function is used with the filename or file path along with the required access modes.

Syntax of `fopen()`

`FILE* fopen(const char *file_name, const char *access_mode);`

Parameters

- *file_name*: name of the file when present in the same directory as the source file. Otherwise, full path.
- *access_mode*: specifies the purpose for which the file is opened (e.g., read, write, append, etc.).

Return Value

- If the file is opened successfully, returns a file pointer to it.
- If the file is not opened, then returns NULL.

// C Program to illustrate file opening

```
#include <stdio.h>
```

```
int main()
```

```
read mode("r")
```

```
{
```

```
FILE* fptr;
```

```
fptr = fopen("/home/shruti/Desktop/1.txt", "r"); // opening the file in read mode
```

```
if (fptr == NULL) {
```

```
printf("The file is not opened. The program will now exit.");
```

```
}
```

else

{

printf("The file is opened. The program will now exit.");

}

return 0;

}

If the file doesn't exist,

```
shruti@shruti-Inspiron-13-5310:~$ gedit ee.c
shruti@shruti-Inspiron-13-5310:~$ gcc ee.c
shruti@shruti-Inspiron-13-5310:~$ ./a.out
shruti@shruti-Inspiron-13-5310:~$ ./a.out
The file is not opened. The program will now exit.
```

If the file exists,

```
shruti@shruti-Inspiron-13-5310:~$ gedit ee.c  
shruti@shruti-Inspiron-13-5310:~$ gcc ee.c  
shruti@shruti-Inspiron-13-5310:~$ ./a.out  
The file is opened. The program will now exit.
```

// C Program to illustrate file opening

```
#include <stdio.h>
```

```
int main()
```

```
write mode("w")
```

```
{
```

```
FILE* fptr;
```

```
fptr = fopen("/home/shruti/Desktop/2.txt", "w"); // opening the file in write mode
```

```
    if (fptr == NULL) {
```

```
        printf("The file is not opened. The program will now exit.");
```

```
    }
```


else

{

printf("The file is opened. The program will now exit.");

}

return 0;

}

If the file 2.txt doesn't exist, then it will create 2.txt



1.txt



allotment.
pdf



allotment.
xls



Lab
programs.
doc



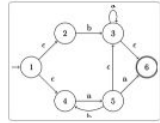
Lab
programs.
pdf



loop
programs.
pdf



T3t9lf.png



VNHmf5.
png

If the file 2.txt doesn't exist, then it will create 2.txt



1.txt



2.txt



allotment.
pdf



allotment.
xls



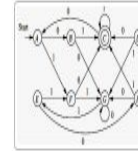
Lab
programs.
doc



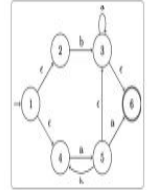
Lab
programs.
pdf



loop
programs.
pdf



T3t9lf.png



VNHmf5.
png

```
shruti@shruti-Inspiron-13-5310:~$ gedit ee.c  
shruti@shruti-Inspiron-13-5310:~$ gcc ee.c  
shruti@shruti-Inspiron-13-5310:~$ ./a.out  
The file is opened. The program will now exit.
```

// C Program to illustrate file opening

```
#include <stdio.h>
```

```
int main()
```

```
append mode("a")
```

```
{
```

```
FILE* fptr;
```

```
fptr = fopen("/home/shruti/Desktop/2.txt", "a"); // opening the file in append mode
```

```
if (fptr == NULL) {
```

```
printf("The file is not opened. The program will now exit.");
```

```
}
```

```
else  
{  
    printf("The file is opened for appending. The program will now exit.");  
}  
return 0;  
}
```

Reading from a data file

- **fscanf()**
- **fgets()**
- **fgetc()**
- **fread()**

fscanf()

Syntax:

```
int fscanf(FILE *stream, const char *format, ...);
```

- **Explanation:**

- fscanf reads formatted input from the given file stream (stream) based on the specified format.
- It returns the number of successfully read items.


```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *file = fopen("input.txt", "r");
```

```
    if (file == NULL) {
```

```
        printf("The file is not opened. The program will now exit.");
```

```
    }
```

```
    int num1, num2;
```

```
    char str[100];
```

```
    fscanf(file, "%d %d %s", &num1, &num2, str); // Using fscanf to read formatted data from the file
```

```
    printf("Read from file using fscanf: %d %d %s\n", num1, num2, str);
```

```
    fclose(file);
```

```
    return 0;
```

```
}
```

Reading from file using fscanf()

input.txt

42 73 HelloWorld

```
shruti@shruti-Inspiron-13-5310:~$ gedit ee.c  
shruti@shruti-Inspiron-13-5310:~$ gcc ee.c  
shruti@shruti-Inspiron-13-5310:~$ ./a.out  
Read from file using fscanf: 42 73 HelloWorld
```

fgets() - Read a line from a file

Syntax:

char *fgets(char *str, int n, FILE *stream);

- **Explanation:**

- fgets reads up to n-1 characters from the file stream stream into the string str.
- It stops reading if a newline character is encountered or n-1 characters are read.
- It appends a null character ('\0') to the end of the string.

```
#include <stdio.h>
```

Reading from file using fgets()

```
int main() {
```

```
    FILE *file = fopen("input.txt", "r");
```

```
    if (file == NULL) {
```

```
        printf("The file is not opened. The program will now exit.");
```

```
    }
```

```
    char buffer[100];
```

```
    fgets(buffer, sizeof(buffer), file); // Using fgets to read a whole line from the file
```

```
    printf("Read from file using fgets: %s", buffer);
```

```
    fclose(file);
```

```
    return 0;
```

```
}
```

input.txt

This is a line of text in the file.

Output:

Read from file using fgets:This is a line of text in the file.

```
shruti@shruti-Inspiron-13-5310:~$ gedit ee.c
shruti@shruti-Inspiron-13-5310:~$ gcc ee.c
shruti@shruti-Inspiron-13-5310:~$ ./a.out
Read from file using fgets: This is a line of text in the file.
```

fgetc() - Read a character from a file

Syntax:

int fgetc(FILE *stream);

- **Explanation:**

- fgetc reads a character from the file stream stream.
- It returns the read character as an unsigned char cast to an int or EOF if the end of the file is reached.

```
#include <stdio.h>
```

```
int main() {
```

Reading from file using fgetc()

```
    FILE *file = fopen("input.txt", "r");
```

```
    if (file == NULL) {
```

```
        printf("The file is not opened. The program will now exit.");
```

```
    }
```

```
    char ch;
```

```
    ch = fgetc(file);    // Using fgetc to read a single character from the file
```

```
    printf("Read from file using fgetc: %c\n", ch);
```

```
    fclose(file);
```

```
    return 0;
```

```
}
```


input.txt

This is a line of text in the file.

```
shruti@shruti-Inspiron-13-5310:~$ gedit ee.c
shruti@shruti-Inspiron-13-5310:~$ gcc ee.c
shruti@shruti-Inspiron-13-5310:~$ ./a.out
Read from file using fgetc: T
```

fread() - Read data from a file

Syntax:

size_t fread(void *ptr, size_t size, size_t no_of_items, FILE *stream);

- **Explanation:**

- fread reads **no_of_items** of data, each **size** bytes long, from the **file stream** into the memory pointed to by **ptr**.
- It returns the number of items successfully read.

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *file = fopen("/home/shruti/Desktop/binary_input.bin", "rb");
```

```
    if (file == NULL) {
```

```
        printf("The file is not opened. The program will now exit.");
```

```
    }
```

```
    int data;
```

```
    size_t elements_read = 1;
```

```
    fread(&data, sizeof(int), elements_read, file); // Using fread to read data from a binary file
```

```
    printf("Read from binary file using fread: %d\n", data);
```

```
    fclose(file);
```

```
    return 0;
```

```
}
```



rb- read binary file

binary_input.bin

2A 00 00

```
shruti@shruti-Inspiron-13-5310:~$ gedit ee.c
shruti@shruti-Inspiron-13-5310:~$ gcc ee.c
shruti@shruti-Inspiron-13-5310:~$ ./a.out
Read from binary file using fread: 807420210
```

- In little-endian, it represents the integer `0x00002A`, which is `42` in decimal.
- In big-endian, it represents the integer `0x2A000000`, which is `807420210` in decimal.

Writing data to a file

- **fprintf()**
- **fputs()**
- **fputc()**
- **fwrite()**

fprintf() - Write formatted data to a file

Syntax:

int fprintf(FILE *stream, const char *format, ...);

- **Explanation:**

- fprintf writes formatted output to the **file stream** based on the specified **format**.
- It returns the number of characters written.

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *file = fopen("/home/shruti/Desktop/output.txt", "w");
```

```
    if (file == NULL) {
```

```
        printf("The file is not opened. The program will now exit.");
```

```
    }
```

```
    int num1 = 42, num2 = 73;
```

```
    char str[ ] = "HelloWorld";
```

```
    fprintf(file, "%d %d %s", num1, num2, str); // Using fprintf to write formatted data to the file
```

```
    printf("Data written to file using fprintf\n");
```

```
    fclose(file);
```

```
    return 0;
```



```
shruti@shruti-Inspiron-13-5310:~$ gedit ee.c  
shruti@shruti-Inspiron-13-5310:~$ gcc ee.c  
shruti@shruti-Inspiron-13-5310:~$ ./a.out  
Data written to file using fprintf
```

Open



output.txt
~/Desktop

Save



1 42 73 HelloWorld

fputs() - Write a string to a file

Syntax:

int fputs(const char *str, FILE *stream);

- **Explanation:**

- fputs writes the null-terminated string str to the file stream stream.
- It returns a non-negative value on success and EOF on failure.

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *file = fopen("/home/shruti/Desktop/output.txt", "w");
```

```
    if (file == NULL) {
```

```
        printf("The file is not opened. The program will now exit.");
```

```
    }
```

```
    char str[ ] = "This is a line of text in the file.";
```

```
    fputs(str, file);    // Using fputs to write a whole line to the file
```

```
    printf("Data written to file using fputs\n");
```

```
    fclose(file);
```

```
    return 0;
```

```
}
```

```
shruti@shruti-Inspiron-13-5310:~$ gedit ee.c  
shruti@shruti-Inspiron-13-5310:~$ gedit ee.c  
shruti@shruti-Inspiron-13-5310:~$ gcc ee.c  
shruti@shruti-Inspiron-13-5310:~$ ./a.out  
Data written to file using fputs
```

Open



output.txt

~/Desktop

Save



1 This is a line of text in the file.

fputc() - Write a character to a file

Syntax:

int fputc(int character, FILE *stream);

- **Explanation:**

- fputc writes the character specified by character to the file stream stream.
- It returns the written character on success or EOF on failure.

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *file = fopen("/home/shruti/Desktop/output.txt", "w");
```

```
    if (file == NULL) {
```

```
        printf("The file is not opened. The program will now exit.");
```

```
    }
```

```
    char ch='A';
```

```
    fputc(ch, file);    // Using fputc to write a character to the file
```

```
    printf("Data written to file using fputc\n");
```

```
    fclose(file);
```

```
    return 0;
```

```
}
```


Open



output.txt
~/Desktop

Save



1 A

```
shruti@shruti-Inspiron-13-5310:~$ gedit ee.c
shruti@shruti-Inspiron-13-5310:~$ gcc ee.c
shruti@shruti-Inspiron-13-5310:~$ ./a.out
Data written to file using fputc
```

fwrite() - Write data to a file

Syntax:

size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);

- **Explanation:**

- fwrite writes nmemb items of data, each size bytes long, from the memory pointed to by ptr to the file stream stream.
- It returns the number of items successfully written.

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *file = fopen("/home/shruti/Desktop/binary_output.bin", "rb");
```

```
    if (file == NULL) {
```

```
        printf("The file is not opened. The program will now exit.");
```

```
    }
```

```
    int data = 42;
```

```
    fwrite(&data, sizeof(int), 1, file); // Using fwrite to write data to a binary file
```

```
    printf("Data written to binary file using fwrite\n");
```

```
    fclose(file);
```

```
    return 0;
```

```
}
```

Open



binary_output.bin

~/Desktop

1 2A 00 00 00

2 |

```
shruti@shruti-Inspiron-13-5310:~$ gcc ee.c
```

```
shruti@shruti-Inspiron-13-5310:~$ ./a.out
```

```
Data written to binary file using fwrite
```

Writing a data to file in append mode

- `fprintf()`
- `fputs()`
- `fputc()`
- `fwrite()`

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *file = fopen("/home/shruti/Desktop/output.txt", "a");
```

```
    if (file == NULL) {
```

```
        printf("The file is not opened. The program will now exit.");
```

```
    }
```

```
    int num1 = 42, num2 = 73;
```

```
    char str[ ] = "HelloWorld";
```

```
    fprintf(file, "%d %d %s", num1, num2, str); // Using fprintf to write formatted data to the file
```

```
    printf("Data written to file using fprintf\n");
```

```
    fclose(file);
```

```
    return 0;
```

fprintf()

```
shruti@shruti-Inspiron-13-5310:~$ gedit ee.c  
shruti@shruti-Inspiron-13-5310:~$ gcc ee.c  
shruti@shruti-Inspiron-13-5310:~$ ./a.out  
Data written to file using fprintf
```

Open



output.txt
~/Desktop

```
1 A42 73 HelloWorld
```



```
#include <stdio.h>
```

```
int main() {
```

fputs()

```
    FILE *file = fopen("/home/shruti/Desktop/output.txt", "a");
```

```
    if (file == NULL) {
```

```
        printf("The file is not opened. The program will now exit.");
```

```
    }
```

```
    char str[ ] = "This is a line of text in the file.";
```

```
    fputs(str, file);    // Using fputs to write a whole line to the file
```

```
    printf("Data written to file using fputs\n");
```

```
    fclose(file);
```

```
    return 0;
```

```
}
```

```
shruti@shruti-Inspiron-13-5310:~$ gedit ee.c  
shruti@shruti-Inspiron-13-5310:~$ gcc ee.c  
shruti@shruti-Inspiron-13-5310:~$ ./a.out  
Data written to file using fputs
```

Open



output.txt

~/Desktop

```
1 A42 73 HelloWorldThis is a line of text in the file.
```

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *file = fopen("/home/shruti/Desktop/output.txt", "a");
```

```
    if (file == NULL) {
```

```
        printf("The file is not opened. The program will now exit.");
```

```
    }
```

```
    char ch='D';
```

```
    fputc(ch, file);    // Using fputs to write a whole line to the file
```

```
    printf("Data written to file using fputc\n");
```

```
    fclose(file);
```

```
    return 0;
```

```
}
```

fputc()

```
shruti@shruti-Inspiron-13-5310:~$ gcc ee.c
```

```
shruti@shruti-Inspiron-13-5310:~$ ./a.out
```

```
Data written to file using fputc
```

Open



output.txt

~/Desktop

```
1 A42 73 HelloWorldThis is a line of text in the file.D
```